Institute of Foundation Models
Mohamed bin Zayed University of Artificial Intelligence

K2-Think: A Parameter-Efficient Reasoning System

Zhoujun Cheng*, Richard Fan*, Shibo Hao*, Taylor W. Killian*,O, Haonan Li*, Suqi Sun*
Hector Ren, Alexander Moreno, Daqian Zhang, Tianjun Zhong, Yuxin Xiong, Yuanzhe Hu, Yutao Xie
Xudong Han, Yuqi Wang, Varad Pimpalkhute, Yonghao Zhuang, Aaryamonvikram Singh, Xuezhi Liang
Anze Xie, Jianshu She, Desai Fan, Chengqian Gao, Liqun Ma, Mikhail Yurochkin, John Maggs
Xuezhe Ma, Guowei He, Zhiting Hu, Zhengzhong Liu*,O, Eric P. XingO

Institute of Foundation Models, Mohamed bin Zayed University of Artificial Intelligence

*Core Contributors (listed alphabetically), OCorresponding Authors

arXiv:2509.07604v3 [cs.LG] 15 Sep 2025

We introduce K2-Think, a reasoning system that achieves frontier performance with just a 32B parameter model — surpassing or matching much larger models such as GPT-OSS 120B and DeepSeek v3.1. Built on the Qwen2.5 base model, our system demonstrates that smaller models can compete at the highest levels through synergistic combination of advanced post-training and test-time computation techniques. Our approach is built on top of six key technical pillars: Long Chain-of-thought Supervised Finetuning, Reinforcement Learning with Verifiable Rewards (RLVR), Agentic planning prior to reasoning, Test-time Scaling, Speculative Decoding, and Inference-optimized Hardware, using only publicly available open-source datasets. K2-Think prioritizes mathematical reasoning, achieving state-of-the-art scores on public benchmarks for open source models, while also maintaining strong performance on other domains such as Code and Science. Our results validate that a more parameter-efficient model like K2-Think 32B can rival state-of-the-art systems through an integrative post-train recipe including long chain-of-thought training and strategic inference-time enhancements, paving the way for more accessible and affordable open-source reasoning systems. We have made K2-Think freely available at k2think.ai demonstrating best-in-class inference speeds, through the Cerebras Wafer-Scale Engine, delivering upwards of 2,000 tokens per second per request.

K2 Think

🤗 K2-Think (Model)	huggingface.co/LLM360/K2-Think
K2-Think (Code)	github.com/MBZUAI-IFM/K2-Think-SFT github.com/MBZUAI-IFM/K2-Think-Inference
K2-Think (Web)	k2think.ai

[0]Correspondence to: {Eric.Xing,Hector.Liu,Taylor.Killian}@mbzuai.ac.ae

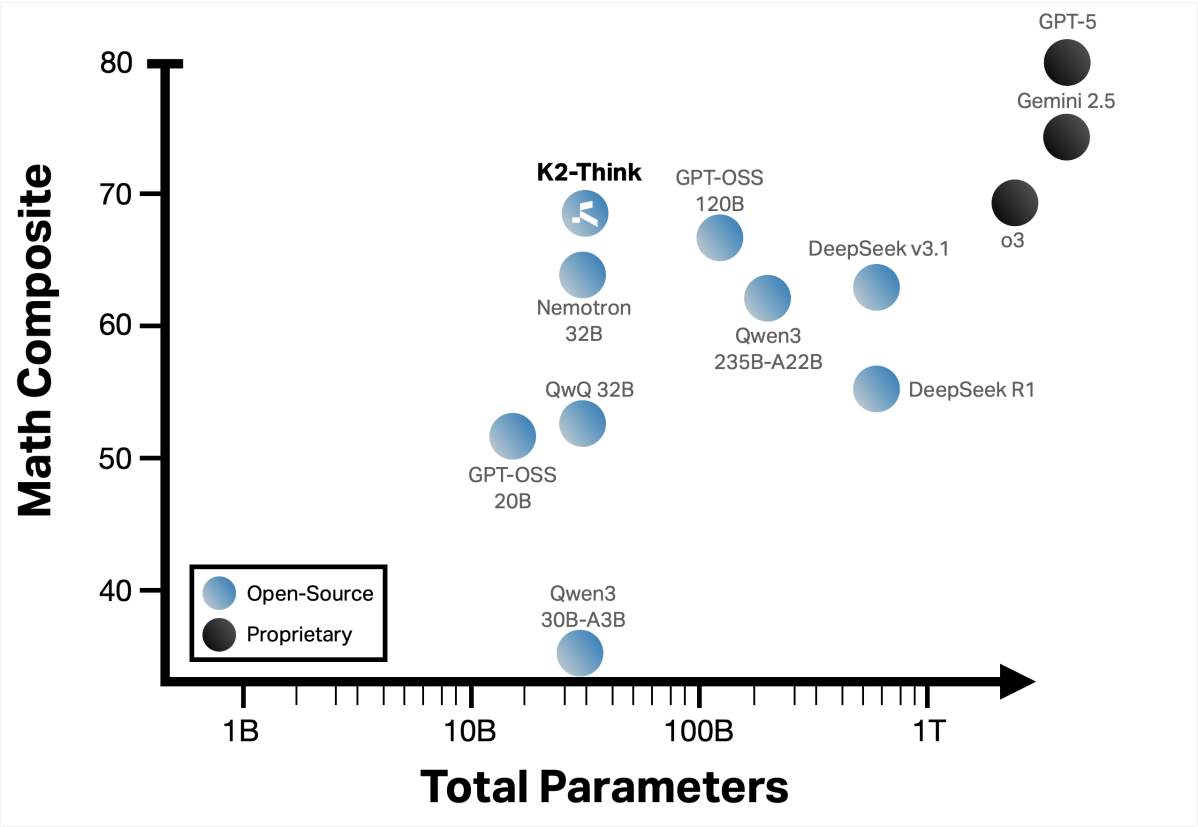

Figure 1: K2-Think exhibits remarkable parameter efficiency, providing comparable or superior performance to frontier reasoning models in complex math domains with an order of magnitude smaller model. The composite score here is the micro-average for each model over four complex math benchmarks, weighted by the number of questions in each benchmark (AIME 2024, AIME 2025, HMMT 2025, and Omni-MATH-HARD; see Section 3 for the benchmark details). Note: parameter counts for proprietary models are speculative.

1 Introduction

Recent advances in frontier reasoning models have highlighted the effectiveness of long chain-of-thought reasoning, enabled by large-scale supervised fine-tuning and reinforcement learning. Systems like OpenAI-O3 (OpenAI, 2025) and Gemini 2.5 (Google DeepMind, 2025) have achieved strong results on competition-level math benchmarks, complex coding tasks, and advanced scientific reasoning datasets, setting new milestones for reasoning-centered language models. These developments have also stimulated further exploration in the open-source community, where researchers have trained competitive reasoning systems with reinforcement learning (Yu et al., 2025; Hu et al., 2025; Wang et al., 2025c) and investigated mechanisms by which RL improves reasoning (Zeng et al., 2025; Yue et al., 2025; Shao et al., 2025; Agarwal et al., 2025b; Wang et al., 2025b).

In this report we introduce K2-Think: a competitive reasoning system built from the open-weight Qwen2.5-32B base model (Yang et al., 2024a). We break down our system into stages, including post-training and test-time, where an integrative recipe involving 6 major technical innovations was introduced over all stages spanning finetuning, reinforcement learning, planning, and hardware optimization to boost the base model reasoning capability, and we evaluated how each stage affects performance. **These components combine to enable a model of merely 32 billion parameters, with modest test-time compute, to match the mathematical reasoning performance of proprietary frontier models**. In fact, K2-Think emerges as the top open-source model for complex math benchmarks matching or exceeding previously leading

models that are orders of magnitude greater in size. Figure 1 presents a plot of the global micro-average of performance (essentially dividing the total number of correct answers by the total number of questions across all test sets) of each model over four challenging math competition tasks with respect to the total number of parameters for each model. The prominent positioning of K2-Think in the top-left visually depicts its superior parameter efficiency, demonstrating that it achieves State-of-the-Art performance among open-source models with a significantly smaller total parameter count. Detailed results and discussion for the benchmarks are presented in Section 3.

More specifically, K2-Think incorporates six key innovations to deliver a strong reasoning system. We first extend the base model with *chain-of-thought capabilities through Supervised Fine-tuning (SFT)*, followed by *Reinforcement Learning with Verifiable Rewards (RLVR)* to strengthen reasoning performance. We then enhance the model with inference-time techniques: *agentic planning* and *test-time scaling* using *Best-of-N sampling*. Finally, we deploy K2-Think with two speed optimizations: *speculative decoding* and Cerebras' Wafer-Scale Engine, an *inference-optimized hardware* system. This final stage enables the model to deliver its powerful chain-of-thought reasoning capabilities with near-instantaneous response times, deployed at speeds upwards of 2000 tokens per second per user request.

With the release of K2-Think, we share our experience and make available an important advancement in open-source language modeling, that aggressive post-training engineering and test-time computation, even with a modest commodity pretrained base model, can significantly boost reasoning capabilities in a cost-effective manner. Prior studies have reported that, in certain regimes, allocating more computation during inference can be more cost-effective than scaling model size (Snell et al., 2025); for recent frontier systems—including OpenAI's o1/o3 (OpenAI, 2024, 2025), DeepSeek-R1 (Guo et al., 2025), Google's Gemini 2.5 (Google DeepMind, 2025), and xAI's Grok4 (xAI, 2025) — model capabilities have been claimed to improve with increased test-time budgets (Ji et al., 2025a; Yang et al., 2025b).

In addition to releasing code and model weights, **we offer K2-Think through a public website and as a production-ready API endpoint.**[1] This allows the community to engage directly with a living system, shifting the emphasis from static artifacts to a deployable, studyable service that can be stress-tested and iterated on in the open. As dynamic inference-time reasoning becomes more complex, our API demonstrates the requirements of sophisticated systems for top performance, and provides an operational deployment delivering robustness, safety, and efficiency under real-world constraints.

In Section 2 we describe the development process and deployment of K2-Think, using the Cerebras Wafer-Scale Engine. Section 3 presents a thorough set of evaluations and ablations that attribute gains across post-training and test-time computation. Section 4 situates our contributions within the literature. We conclude in Section 5 with a summary overview, discuss our motivations for deploying this model, and chart future directions for extending reasoning performance with openly released models and deployment-ready systems.

2 K2-Think Development

We initiate K2-Think's development to study a complete post-training recipe for enhanced reasoning and establish best practices for extending our in-house foundation models. Throughout this study, we seek to validate published best practices as well as test original test-time computation ideas.

We choose to fine-tune a 32B-scale base model for K2-Think, as:

(1) it allows for fast iteration while providing strong base capabilities and

(2) its size suits both research and consumer computation frameworks.

[1] available upon request

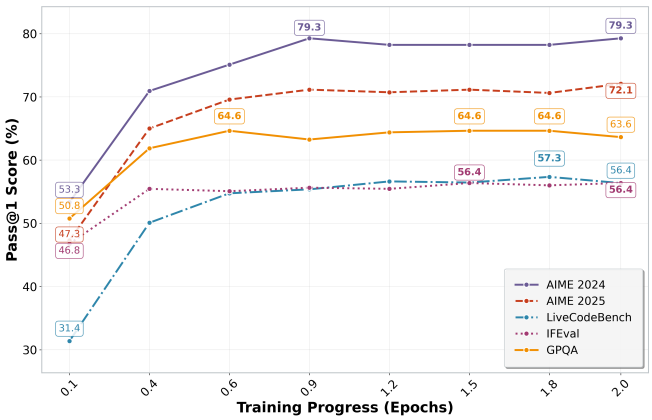

Figure 2: Pass@1 performance over training. Pass@1 of K2-Think-SFT across five benchmarks; the x-axis is training progress (epochs), the y-axis is pass@1 score.

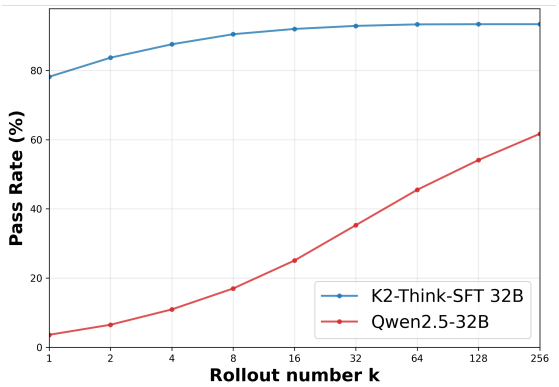

Figure 3: Pass@k on AIME2024. Pass@k of K2-Think-SFT and the Qwen-2.5 32B base model; the x-axis is the number of rollouts per question, the y-axis is pass rate.

Specifically, we selected Qwen2.5-32B as it is not tuned for reasoning, allowing us to fully validate our recipe's effectiveness.

2.1 Phase 1: Supervised Fine Tuning

The initial stage of K2-Think development constitutes supervised fine-tuning (SFT) of the base model using curated long chain-of-thoughts (CoT), establishing the first pillar of our complete reasoning system. This follows the paradigm introduced by DeepSeek in the development of their R1 model (Guo et al., 2025). This phase of training serves to provide guidance to the pre-trained base language model for generating structured responses to complex queries. Additionally, the model is trained to adopt an expected output format in which the model's reasoning process is made clear prior to producing an answer. By providing a token-by-token supervisory signal through extended CoT, the base model's intrinsic computation capabilities are expanded substantially (Wei et al., 2022; Schuurmans et al., 2024).

Our SFT phase[2] uses the existing `AM-Thinking-v1-Distilled` dataset,[3] composed of CoT reasoning traces and instruction/response pairs, with prompts drawn from tasks spanning mathematical reasoning, code generation, scientific reasoning, instruction following, and general chat (Ji et al., 2025b; Tian et al., 2025). In what follows, we will refer to this supervised fine-tuned model as **K2-Think-SFT**.

2.1.1 Observations

Our SFT experiments on Qwen2.5-32B yield several practical insights. Of particular note, we conduct a step-wise evaluation of K2-Think-SFT across five representative benchmarks. As shown in Figure 2, performance improves rapidly within the first third of training (roughly 0.5 epoch), particularly on mathematics benchmarks (AIME 2024 and AIME 2025). After this sharp initial gain, most benchmarks plateau, with AIME 2024 stabilizing around 79.3% pass@1 and AIME 2025 around 72.1%. GPQA and IFEval continue to exhibit modest upward trends, while LiveCodeBench shows a slower but steady improvement up to 56.4%. We observe that our SFT phase has reached convergence, with the model exhibiting diminishing returns to continued training on the dataset.

Apart from pass@1 scores, we also use pass@k to quantify reasoning performance under a fixed sampling budget k. Interpreting the pass@k curve as a capability boundary, we evaluate K2-Think-SFT. In Figure 3, assessing performance on AIME2024, our SFT model dominates the base model across sampling budgets. The SFT curves saturate near 93.3% by $k \approx 128$, whereas the base model continues to improve but remains

[2] Code, based on LLaMA-Factory, for SFT can be found at https://github.com/MBZUAI-IFM/K2-Think-SFT
[3] https://huggingface.co/datasets/a-m-team/AM-Thinking-v1-Distilled

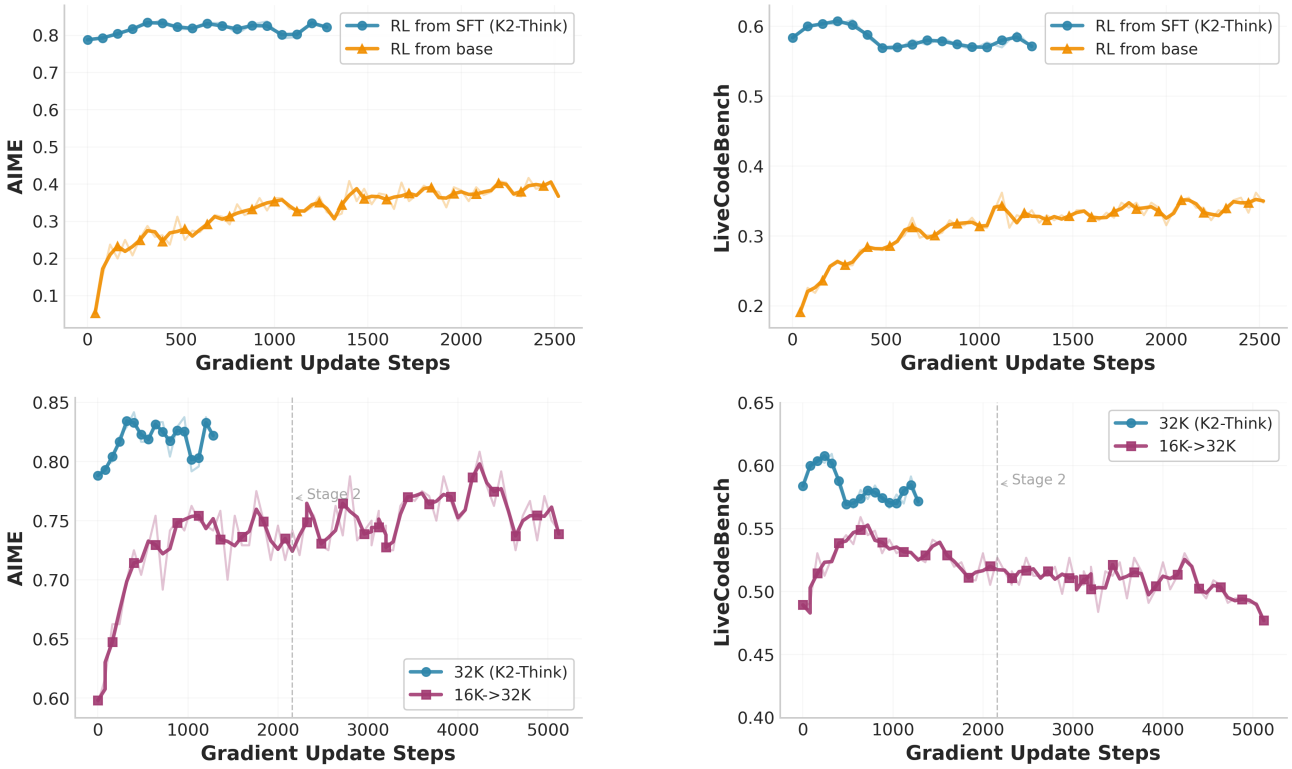

Figure 4: Ablation Studies on Multi-stage Training and RL from Base Models. (top): RL from base models achieves much faster performance gains compared to RL from SFT models. However, a substantial performance gap remains, suggesting that SFT enhances the model's score at the cost of slower subsequent improvement and increased susceptibility to collapse during RL. **(bottom)**: reducing K2-Think-SFT maximum response length significantly impacts performance. Multi-stage training (16,000 to 32,000) struggles to recover original performance, even with prolonged training.

well below that plateau. The growth in K2-Think-SFT performance as the sampling budget grows suggests there remains an opportunity for improvement during the following RL stage.

2.2 Phase 2: Reinforcement Learning with Verifiable Rewards

Following the SFT stage, we perform Reinforcement Learning with Verifiable Rewards (RLVR) to train K2-Think to excel in domains with verifiable outcomes, which constitutes the second pillar of our full reasoning system. RLVR reduces the complexity and cost of preference-based alignment via RLHF (Casper et al., 2023) by directly optimizing for correctness of model generations.

For K2-Think's RLVR, we use the Guru dataset (Cheng et al., 2025), which was curated to extend open-source reasoning models to verifiable domains beyond Math and Code. We leverage all six domains from the Guru dataset, comprising nearly 92,000 verifiable prompts that cover Math, Code, Science, Logic, Simulation, and Tabular tasks. We refer interested readers to the Guru paper for the detailed dataset curation, including de-duplication, reward designs, and filtering. Our RLVR implementation is built on the `verl` library (Sheng et al., 2025) with the GRPO algorithm (Shao et al., 2024).

2.2.1 Observations

In this subsection we provide a retrospective set of observations that serve as motivation for future development.

Starting from a strong SFT checkpoint yields better performance but limits RL gains. While RL consistently improves K2-Think-SFT performance across internal evaluations and public benchmarks, the absolute improvements were modest. As a comparative experiment, we also train a model with the same RL recipe and Guru data directly from the Qwen2.5-32B base. Figure 4 (top) demonstrates that RL training from the base model achieves nearly 40% improvement on AIME 2024 over the training course, while RL from K2-Think-SFT yields only 5% improvement. This validates that stronger SFT checkpoints leave less room for RL refinement, consistent with findings from Liu et al. (2025b) regarding the relationship between SFT scope and subsequent RL effectiveness. Also, we notice RL training from the SFT checkpoint exhibits early plateauing and even degradation. We suspect that heavily "SFTed" models become constrained in their ability to explore alternative reasoning strategies during RL training, limiting the policy's capacity for meaningful adaptation.

Multi-stage RL training with reduced initial context length degrades performance. Many concurrent research efforts employ multi-stage training as implicit curriculum learning (An et al., 2025; Liu et al., 2025a; Rastogi et al., 2025), incrementally increasing context length. This accelerates early training while the model develops competency and then allows the model in later stages to handle more difficult questions with the extended context. We test this approach by first constraining model output to 16,000 tokens during initial RL training from K2-Think-SFT, then expanding to 32,000 tokens (this is the maximum length seen during the SFT stage) for continued training. As shown in Figure 4 (bottom), this multi-stage approach failed to match even the baseline SFT model performance. Cutting the maximum length below the SFT training configuration yields substantially lower performance. This negative result undermines the original motivation for multi-stage training to achieve on-par or better performance with shorter responses to save inference tokens. We suspect that reducing context length below the SFT training regime (32k $\rightarrow$ 16k $\rightarrow$ 32k) disrupts the model's established reasoning patterns as we do not perform any additional data filtering to correspond to this multi-stage training. However, we do not evaluate expanding beyond the SFT context length (e.g., 32k $\rightarrow$ 48k), as implemented in Polaris (An et al., 2025), which may still provide benefits.

2.3 Phase 3: Test-time Improvement

To further enhance K2-Think performance, we develop a test-time scaffolding that implements existing methods as well as integrates an original approach to provide structured input to our post-trained reasoning model. This subsection details two specific aspects of this scaffolding: agentic planning before reasoning, namely "Plan-Before-You-Think", and test-time scaling using Best-of-N sampling. These two techniques are pillars three and four of the complete K2-Think system.

A diagram mapping the flow of information from the input provided, down to the final response, is illustrated in Figure 5. First, the prompt is restructured to outline a high-level plan, highlighting relevant concepts. This augmented prompt is then passed through the K2-Think model, generating multiple responses. Finally, a pairwise comparison between candidate responses surfaces the best generation as the final output of our reasoning system. The remainder of this section provides details of how we set-up and implement each of these components.[4]

"Plan-Before-You-Think". The first procedure of K2-Think's test-time computation is the introduction of a planning agent. In our current system implementation, we simply ask the agent to extract key concepts from the query, and create a high-level plan from them. The generated plan is appended alongside the original query, and provided to the K2-Think model. K2-Think's planning agent is simply implemented via prompting an instruction-tuned Language Model. We restrict this "Plan-Before-You-Think" procedure

[4]Code for K2-Think test-time improvements is at: https://github.com/MBZUAI-IFM/K2-Think-Inference

Figure 5: Schematic overview of how K2-Think generates responses via our test-time computation scaffold. A user query is first input to an external model which generates a high-level plan to provide a structured prompt to our K2-Think model. We then sample 3 responses, using an external model to select the best which is then provided as output.

from providing direct answers or any reasoning trace. This deliberation phase, prior to any "thinking" by our reasoning model, has some basis in psychology and cognitive science. Planning and reasoning can be considered dual processes of human cognition and decision making (Evans, 2010) where planning is considered a meta-thinking process developing some structure to help guide one's thoughts.

Best-of-N (BoN) sampling. Best-of-N sampling, sometimes called repeated sampling, is a method where an LLM generates N independent outputs for a given prompt, and a reward model (or verifier) chooses the best one according to some metric—such as accuracy, coherence, or alignment with human preference (Stiennon et al., 2020; Nakano et al., 2021). This strategy effectively explores multiple possibilities and picks the most promising completion.

Implementation-wise, we pick the answer by comparing the answer candidates pairwise, discarding the one that an independent LLM judges to be worse. In K2-Think, we finally adopt $N = 3$, which provides a reasonable improvement with low cost.

2.3.1 Observations

At inference time, we explore several approaches to enhance the K2-Think model's performance. We begin with simple engineering adjustments but soon discover that minor changes to our test-time computation procedures significantly impact overall performance.

We experiment with temperature tuning, iterating through a list of temperatures from 0.1 to 1.0, but find the overall improvement to be insignificant, leading us to use a temperature of 1.0 for all future runs. We also conduct extensive prompt engineering, trying over 30 different system prompts that utilized techniques like few-shot learning (Brown et al., 2020), role-playing (Kong et al., 2023), and situational prompting. However, we observe negligible gains.

More sophisticated test-time scaling methods are also tried following Sharma (2024). We test several standard approaches including re2 (ReRead) (Xu et al., 2024), self-consistency (Wang et al., 2023b), CoT with reflection (Shinn et al., 2023), and Mixture of Agents (MoA) (Wang et al., 2024b). Among these, Best-of-N (BoN) and MoA yield the most notable improvements. While MoA delivers marginally better performance, its significantly higher computational cost lead to the selection of BoN for the final K2-Think system. A different, more experimental approach involving Reinforcement Learning with rewards drawn from self-certainty signals (Zhao et al., 2025) is also tested, but it does not lead to any improvement of the post-trained model's performance.

2.4 Deploying K2-Think

We deploy K2-Think on Cerebras Wafer-Scale Engine (WSE) systems, leveraging the world's largest processor and speculative decoding (Leviathan et al., 2023) to achieve unprecedented inference speeds for the reasoning system, making up the final two pillars of K2-Think. The WSE delivers approximately 2,000 tokens per second, representing a 10 times improvement over the nominal 200 tokens per second observed

on typical deployment environments on a regular cloud provider. This dramatic speed-up fundamentally transforms the practical usability of long chain-of-thought reasoning.

Consider a typical complex reasoning task that generates a 32,000 token response, which is common for challenging mathematical proofs or multi-step coding problems. On the WSE, 32,000 token generation is completed in just 16 seconds, maintaining user engagement, enabling true back-and-forth problem solving, and providing a near real-time chat experience.

The performance advantage comes from the unique architecture of WSE. WSE keeps all model weights in massive on-chip memory, leveraging 25 Petabytes per second of on-chip memory bandwidth. Since auto-regressive models generate tokens serially, memory bandwidth can be a significant bottleneck during inference. By integrating greater compute, memory, and memory bandwidth in a single device, wafer-scale technology enables industry-leading inference speed for generative models.

This efficiency proves especially critical for our test-time computation approach and agent-based reasoning workflows. When performing best-of-3 sampling, the system must wait for all three responses to complete before LLM evaluation can select the optimal solution. Further, multi-step reasoning pipelines that require sequential calls for planning and generation suffer from cumulative delays. The WSE's low-latency inference keeps these workflows interactive, preventing the cascade of delays that would otherwise render complex reasoning tasks impractical.

The difference between waiting minutes versus seconds for each interaction fundamentally transforms the user experience from batch processing to interactive reasoning. This deployment ensures that K2-Think provides not just frontier reasoning capabilities but also the responsiveness required for practical, real-world applications, making sophisticated AI reasoning truly accessible for interactive use cases. We invite everyone to experience our K2-Think system, powered by Cerebras' WSE, via API and through `k2think.ai`.

3 K2-Think Evaluation

We evaluate K2-Think in comparison with frontier models, both open-weight and proprietary, among a class of challenging reasoning benchmarks focused on Math, Code and Science. We design these evaluations to demonstrate that K2-Think, despite only having 32B parameters and fairly modest test-time computation, pushes the frontier of open-source reasoning models. In particular we find that K2-Think is highly capable for complex Math tasks, as shown in Table 1. In total we evaluate K2-Think on the following benchmarks:

- Math
  - **AIME 2024** (MAA, 2024), **AIME 2025** (Ye et al., 2025b): The 2024 and 2025 editions of the American Invitation Mathematics Examination (AIME), with each year featuring 30 questions that have integer answers.
  - **HMMT25** (Balunović et al., 2025): This dataset, used as part of the MathArena benchmarking suite, is drawn from the Harvard-MIT Mathematics Tournament February 2025 competition, featuring 30 questions drawn from the subject areas of Algebra+Number Theory, Combinatorics, and Geometry.
  - **Omni-MATH-HARD** (Omni-HARD, Gao et al. (2024)): We use the most difficult subset of the Omni-MATH dataset, featuring questions sampled from competitive mathematics competitions at the Olympiad level from several countries, retaining only those problems that are rated as the top 2 difficulty levels (9.0 and 10.0). This set has 173 questions, a much larger competition math benchmark, and perhaps the most compelling one.

- A global micro-average (Micro-Avg.) is obtained by dividing the total number of correct answers by the total number of questions across all datasets.

- Code
  - **LiveCodeBench** (LCBv5, Jain et al. (2024)): A collection of 599 programming challenge problems aggregated from online platforms. We use queries aggregated between July 1, 2024 and February 1, 2025 (v5).
  - **SciCode** (Tian et al., 2024): SciCode evaluates a model's ability to generate code for solving 65 realistic scientific research questions, covering 16 subdomains from Physics, Math, Material Science, Biology, and Chemistry. We report scores from the version of the benchmark where background knowledge is included within the prompt. Since SciCode already performs a complex, multi-step planning phase in collating this information we do not run our "Plan-Before-You-Think" step during our evaluation of K2-Think on this baseline. All evaluation results include sub-problem and full-problem accuracies in Table 1.

- Science
  - **GPQA-Diamond** (GPQA-D, Rein et al. (2023)): This benchmark is comprised of 198 "Google-proof" advanced multiple-choice questions written by experts from biology, physics, and chemistry.
  - **Humanity's Last Exam** (HLE, Phan et al. (2025)): Humanity's Last Exam was developed by subject-matter experts and consists of 2158 multiple-choice and short-answer questions with solutions that are unambiguous and easily verifiable, but cannot be quickly answered via internet retrieval.

We measure the performance of K2-Think in comparison to frontier reasoning models, both open-source {Qwen3-30B-A3B (Yang et al., 2025a), GPT-OSS 20B (Agarwal et al., 2025a), QwQ-32B (Team, 2025), OpenReasoning-Nemotron-32B (NVIDIA, 2025), DeepSeek R1 (Guo et al., 2025), DeepSeek-v3.1 (Thinking) (DeepSeek, 2025), GPT-OSS 120B (Agarwal et al., 2025a), Qwen3-235B-A22B (Thinking) (Yang et al., 2025a)} and proprietary {GPT-5 (High) (OpenAI, 2025), Gemini-2.5 (Pro) (Google DeepMind, 2025), o3 (High) (OpenAI, 2025)} to adequately assess the advancements made by our post-training and test-time computation scaffold. We use a standardized evaluation methodology across all benchmarks and models. The maximum generation length is set to 64,000 tokens, sampling temperature is fixed at 1.0, top-p is 0.95 and the stop token is `</answer>`. Each benchmark result reported in Table 1 is the average of 16 independent pass@1 evaluations.

K2-Think excels in competition math questions. The evaluation results are summarized in Table 1. K2-Think, a 32B-parameter model, exhibits a micro-average score of **67.99** across all math questions. This result is particularly noteworthy when compared to other models of similar or slightly larger size, such as GPT-OSS 20B (m-avg. 52.50), Qwen3-30B-A3B (m-avg. 33.08), and OpenReasoning-Nemotron-32B (m-avg. 65.78). The results clearly show that **K2-Think surpasses these models by a significant margin**. Furthermore, K2-Think's performance is not only dominant within its size class but also highly competitive with models that are orders of magnitude larger. Its math score also surpasses the larger models, including the two state-of-the-art open source models: DeepSeek V3.1 671B (m-avg. 64.43) and GPT-OSS 120B (m-avg. 67.20). Notably, K2-Think performs well on Omni-MATH-HARD (60.73), which contains the most difficult questions across competitions. **Such performance places K2-Think at the top of all open source models on math reasoning**, and is close to strong proprietary models such as o3 High, showing that K2-Think excels in the most challenging questions.

Benchmarks →	Math					Code		Science	
Models ↓	AIME 2024	AIME 2025	HMMT25	Omni-HARD	Micro-Avg.	LCBv5	SciCode (sub/main)	GPQA-D	HLE
K2-Think	90.83	81.24	73.75	60.73	67.99	63.97	39.2 / 12.0	71.08	9.95
GPT-OSS 20B	76.88	74.58	69.38	41.51	52.50	73.22	37.9 / 9.0	65.45	11.23
Qwen3-30B-A3B	70.63	58.14	23.54	23.87	33.08	42.20	28.5 / 4.8	58.91	6.14
Nemotron 32B	87.09	82.71	67.29	58.88	65.78	57.79	37.1 / 11.4	74.98	12.26
QwQ-32B	79.38	69.17	51.46	46.93	53.69	65.22	36.9 / 11.5	66.24	9.98
GPT-OSS 120B	89.58	84.59	81.88	57.76	67.20	74.53	38.8 / 11.0	77.04	18.58
Qwen3 235B-A22B	86.68	75.43	61.88	56.91	62.99	56.64	39.3 / 10.9	65.55	14.23
DeepSeek V3.1†	91.87	82.49	83.54	53.22	64.43	66.59	38.2 / 11.7	79.46	8.40
DeepSeek R1†	74.38	65.21	47.08	51.33	55.06	61.01	36.7 / 11.5	71.08	8.50 *
o3 High	92.26	86.58	80.80	59.39	68.68	73.30	41.7 / 11.9	81.30	22.34
Gemini2.5 Pro	87.24	85.75	74.18	69.36	73.82	58.24	45.1 / 15.4	84.51	19.93
GPT-5 High	94.78	92.15	91.79	73.61	80.21	82.68	41.3 / 12.4	85.96	28.63

Table 1: Benchmark performance comparison of K2-Think against open-source (top) and proprietary (bottom) frontier models. All metrics are reported as percentages. **We find that K2-Think is especially strong on challenging Math benchmarks while also maintaining respectable performance on Code and Science.** Values marked with * are directly taken from published results. All other reported values are avg@16 accuracy of generated answers, evaluated locally or through paid API access. From these results, we see that our K2-Think system with only 32B parameters approaches or exceeds the performance of the frontier models that are orders of magnitude larger. *† showing results for the original DeepSeek R1 and V3.1. Since the performance of DeepSeek R1-0528 is similar to V3.1, we do not report it separately.*

K2-Think is versatile in Science and Coding domains. The evaluation results also show that K2-Think demonstrates a robust and competitive capability in both coding and scientific domains, solidifying its position as a versatile model. On coding benchmarks, K2-Think achieves a score of **63.97** on LiveCodeBench, significantly outperforming its similarly sized peers, including GPT-OSS 20B (42.20) and Qwen3-30B-A3B (36.9). This performance also surpasses the larger Qwen3 235B-A22B (56.64). When compared to larger models, **K2-Think shows parity and even superiority in certain metrics**: it achieves 39.2 on the SciCode benchmark (sub-problems), making it a close second compared with Qwen3 235B-A22B (39.3). On scientific reasoning, our system's performance on the GPQA-Diamond benchmark is 71.08, superior to most open-source models except OpenReasoning-Nemotron-32B (74.98), and GPT-OSS 120B (77.04). While its HLE score of 9.95 is not the highest, it remains respectable and indicative of a broad knowledge base. This combination of strong performance across diverse domains argues that K2-Think is not merely a specialist but a versatile model capable of tackling a wide range of analytical and knowledge-intensive tasks with high efficacy.

Beyond the preliminary conclusions shared in this report, our team is continuing to perform additional analyses and comparisons between K2-Think and a more complete set of competing models and reasoning benchmarks. It is however clear that K2-Think presents an advancement in open-source reasoning systems. With a 32B parameter model, and a moderate amount of test-time compute, our system provides comparative performance to models significantly larger (see Figure 1 for a visual depiction). This level of parameter efficiency, in terms of benchmark performance is a notable achievement, specifically among complex mathematics reasoning tasks.

Component Analysis of K2-Think Test-Time Computation In order to analyze the individual contribution of each element of the test-time computation procedure to the final performance of K2-Think, we conduct an analysis where we implement each procedure in isolation on top of the post-trained checkpoint. That is, after performing both SFT and RL, we apply only the prompt restructuring via high-level planning or best-of-3 re-sampling and verification during evaluation. To simplify the discussion, we present this component analysis only using the four Math benchmarks however the overall insights are consistent across all other benchmarks.

	AIME 2024	AIME 2025	HMMT25	Omni-MATH-HARD
SFT+RL Checkpoint	86.26	77.72	66.46	56.74
+ Plan only	85.21	81.04	71.87	58.97
+ Bo3 only	90.77	81.22	71.16	59.47
+ Plan + Bo3 (K2-Think)	90.83	81.24	73.75	60.73

Table 2: Component analysis of the test-time computation procedures used to improve from our post-training checkpoint in the development of our final K2-Think system. The greatest gains come from Best-of-3 sampling, further improvement is seen after combining with high-level planning.

Model	AIME 2024		AIME 2025		HMMT25		Omni-HARD		LCBv5		GPQA-D	
SFT+RL Checkpoint	21,482		25,262		29,136		34,042		13,589		14,998	
K2-Think	20,040	(-6.72%)	24,266	(-3.94%)	27,030	(-7.23%)	30,050	(-11.73%)	12,166	(-10.53%)	14,680	(-2.12%)
Qwen3-235B-A22B	29,896		34,541		39,767		45,701		27,716		20,007	
GPT-OSS 120B	15,971		19,151		25,566		35,021		7,389		11,281	
DeepSeek v3.1	12,364		15,143		19,073		24,841		6,158		6,592	

Table 3: An analytical comparison of the average number of tokens used between the full K2-Think system and the post-training checkpoint. After implementing our test-time computation scaffold, our response length decreases on average, with the percentage of reduction included in the shaded cells. We also compare to the average number of tokens generated by top-performing open-weight models, showing better efficiency than Qwen3-235B-A22B and similar to GPT-OSS 120B.

The component analyses presented here are executed in the same fashion as the comparative baselines contained in Table 1. All results presented in Table 2 are averaged over 16 independent runs with the same settings as presented previously. We see in this analysis that the majority of the improvement over the post-trained checkpoint is afforded via Best-of-N scaling, using only 3 sampled generations per prompt. In isolation, the performance benefit of re-structuring the input prompt with a high-level plan also contributes an improvement to performance but with lesser effect. However, in combination with Best-of-N scaling the overall test-time procedure offers significant gains, offering 4-6 percentage points of improvement across all benchmarks.

"Plan-Before-You-Think" Reduces Response Lengths With the complete K2-Think system, we require a model to create a plan before thinking. While we have shown this procedure to positively affect reasoning performance, the expansion of the prompt might cause more tokens used when formulating an answer. However, we found the opposite to be true. We report the average number of tokens K2-Think generated in the final response before and after implementing test-time computation in Table 3 for select benchmarks evaluated across Math, Code and Science domains, comparing the K2-Think post-training checkpoint and the full system. The inclusion of this plan achieves two benefits: response quality improves and there is a reduction in the number of tokens used, by up to nearly 12% in comparison to the post-training checkpoint. Thus, by conducting planning before reasoning, K2-Think provides more concise answers.

We also compare the average number of tokens used among the best performing open-weight models. We see that the K2-Think responses are much shorter than Qwen3-235-A22B and are in a range similar to the responses from GPT-OSS 120B in mathematical reasoning. When comparing response lengths for the code and science domains, K2-Think are still shorter than Qwen3-235-A22B, but longer than GPT-OSS 120B.

3.1 Red-teaming K2-Think

Ensuring the safe operation of a model is essential for its open release. To this end, we systematically evaluate K2-Think against adversarial prompts, harmful content, and robustness stress tests using

established public safety benchmarks (Lin et al., 2024). For each benchmark, we sample 100 test cases and report a *safe score*, where higher values indicate stronger safety performance. To provide a clear picture of real-world risks, we consolidate results into four key aspects that capture the practical safety surfaces most relevant in deployment:

1. **High-Risk Content Refusal** — ability to reject direct requests for unsafe or harmful outputs.
2. **Conversational Robustness** — maintaining safe behavior consistently across multi-turn dialogues.
3. **Cybersecurity & Data Protection** — resilience against information leakage, prompt extraction, and cyberattack assistance.
4. **Jailbreak Resistance** — robustness to adversarial attacks designed to bypass safeguards.

This framework provides a clearer operational safety profile and guides targeted mitigations.

High-Risk Content Refusal We first check the model's reliability in rejecting unsafe requests. Evaluation spans complementary datasets covering harmful instructions (**Do-Not-Answer** (Wang et al., 2023c), **HarmBench** (Mazeika et al., 2024)), physical harm scenarios (**PhysicalSafety** (Bianchi et al., 2023)), basic safety checks (**SimpleSafetyTests** (Vidgen et al., 2023)), toxic content generation (**ToxiGen** (Hartvigsen et al., 2022; Hosseini et al., 2023)), commonsense safety (**CoNA** (Bianchi et al., 2023)), and harmful Q&A (**HarmfulQ** (Shaikh et al., 2023)).

Dataset	Score
Do-Not-Answer	0.88
HarmBench	0.56
PhysicalSafety	0.49
SimpleSafetyTests	0.95
ToxiGen	0.97
CoNA	0.97
HarmfulQ	0.99
Macro-average	**0.83**

Table 4: High-risk content refusal results across safety datasets. The model achieves near-perfect performance on four of seven tasks, with clear improvement opportunities on HarmBench and PhysicalSafety.

The results of this analysis are featured in Table 4. K2-Think demonstrates extensive ability to avoid generating high-risk content as measured by near-perfect scores in 4 out of 7 benchmarks. Of the remaining 3 benchmarks in this aspect of safety evaluation, HarmBench and PhysicalSafety reveal a weakness in our system toward recognizing cyber or physical risks. We are actively working to improve our system along these dimensions of risk in its public facing deployment.

Conversational Robustness Next, we assess refusal consistency across multi-turn adversarial dialogues using **DialogueSafety** (Dinan et al., 2019), **HH-RLHF** (Bai et al., 2022), and **DICES350** (Aroyo et al., 2023) for dynamic dialogue manipulations.

We see in Table 5 that K2-Think is especially robust to sustained adversarial dialogues and repeated efforts to ellicit harmful behaviors from our reasoning system. Here, K2-Think is near perfect at maintaining refusal consistency on both the DialogueSafety and HH-RLHF benchmarks.

Cybersecurity & Data Protection & Prompt Extraction We evaluate resilience against data leakage and misuse with **PersonalInfoLeak** (Li et al., 2023) (privacy leakage), **CyberattackAssistance** (Bhatt et al., 2023) (hacking assistance), and **PromptExtractionRobustness** (Toyer et al., 2023) (system prompt extraction).

We see in Table 6 that K2-Think is able to resist attempts to extract personally identifying information while unfortunately exhibiting some susceptibility to revealing the system prompt and aiding in devising cyberattacks. This indicates an opportunity to further tune our reasoning system for improved resilience.

Dataset	Score
DialogueSafety	0.99
HH-RLHF	0.95
DICES350	0.73
Macro-average	**0.89**

Table 5: Conversational robustness results across dialogue safety datasets. The model exhibits notable robustness to multi-turn adversarial attempts to produce harmful outputs, with particular strength on DialogueSafety and room for improvement on DICES350.

Dataset	Score
PersonalInfoLeak (few-shot)	0.86
CyberattackAssistance	0.47
PromptExtractionRobustness	0.35
Macro-average	**0.56**

Table 6: Cybersecurity, data protection, and prompt extraction results. The model demonstrates robustness against leaking personal information, with significant room for improvement on cyberattack assistance prevention and prompt extraction robustness.

Jailbreak Resistance Finally, we evaluate various adversarial attack strategies: hidden triggers (**LatentJailbreak** (Qiu et al., 2023)), prompt redirection (**PromptInjection** (Liu et al., 2023b)), instruction overrides (**Gandalf Ignore** (Schulhoff et al., 2023)), role-play attacks (**DAN** (Shen et al., 2023)), cross-lingual exploits (**Multilingual** (Wang et al., 2023a)), grammatical perturbations (**Tense Change** Lin et al. (2024)), adversarial demonstrations (**Few-Shot Attack** (Wei et al., 2023b)), bias-driven attacks (**One-Sided Statement** (Liu et al., 2023a)), identity manipulation (**Persona Modulation** (Shah et al., 2023)), and direct refusal bypasses (**Refusal Suppression** (Wei et al., 2023a)).

Dataset	Score
Few-Shot Attack	0.96
Gandalf Ignore	0.87
Tense Change	0.84
Multilingual	0.83
PromptInjection	0.77
One-Sided Statement	0.77
Refusal Suppression	0.76
Persona Modulation	0.59
Do-Anything-Now	0.43
LatentJailbreak	0.37
Macro-average	**0.72**

Table 7: Jailbreak resistance results across adversarial prompt techniques. The model demonstrates mixed resilience, with strong performance against direct attacks and vulnerabilities to indirect methods.

K2-Think's jailbreak resistance results (shown in Table 7) demonstrate a mixture of resilience and susceptibility to various adversarial prompt strategies. K2-Think exhibits strong performance when attacks are immediately apparent but shows an apparent weakness to indirect attacks. This lack of generalized robustness to adversarial jailbreaking attempts illustrates a need to thoroughly improve our publicly deployed reasoning system.

Overall Results Across all four dimensions, results are aggregated into a single Safety-4 macro score, computing the average from the four analyses performed as part of our safety testing of K2-Think. The macro average of each of the four analyses are included in Table 8.

Overall, K2-Think achieves a **Safety-4 macro score of 0.75**, indicating a solid baseline of safety with strong performance in refusing harmful content and maintaining consistent behavior in conversations. At the same time, we recognize that further work is required to strengthen **cybersecurity defenses, jailbreak robustness, and refusal calibration**. While establishing a solid baseline, we acknowledge clear opportunities to improve the safety of our reasoning system. Addressing these areas is an active priority in our roadmap to further improve K2-Think under adversarial conditions.

Safety Aspect	Macro-Avg Score
High-Risk Content Refusal	0.83
Conversational Robustness	0.89
Cybersecurity & Data Protection	0.56
Jailbreak Resistance	0.72
Safety-4 Macro (avg)	**0.75**

Table 8: Overall Safety-4 results which is a composite score of the four safety surfaces evaluated in this broad analysis. The macro score of 0.75 indicates that K2-Think establishes a solid safety profile with specific strengths in harmful content refusal and maintaining consistent behavior in conversations.

4 Related Work

Extending base language model capabilities via SFT Supervised fine-tuning (SFT) has become a widely used post-training method to extend the capability boundary of Large Language Models (Ouyang et al., 2022; Dubey et al., 2024; Guo et al., 2025; Bercovich et al., 2025). Early SFT work primarily focused on task specialization, adapting foundational models to specific NLP benchmarks like text classification or translation on narrowly-defined datasets (Liu et al., 2019; Raffel et al., 2020). This paradigm shifted significantly with the rise of large-scale instruction tuning; the goal evolved from single-task mastery to creating general-purpose assistants capable of following diverse human commands (Wei et al., 2021; Ouyang et al., 2022; Taori et al., 2023). More recently, SFT has pivoted towards enhancing complex reasoning on diverse downstream tasks like math, code, and science (Hui et al., 2024; Yang et al., 2024b; Abdin et al., 2025; Liu et al.). Some approaches focus on scale, constructing massive datasets of reasoning traces to instill robust, long-chain-of-thought capabilities in models (Guha et al., 2025; Tian et al., 2025; Liu et al., 2025b). In contrast, other methods demonstrate that meticulously curated, high-quality data can also endow LLMs with expert-level reasoning in domains like math (Ye et al., 2025a; Muennighoff et al., 2025). Building on the above, our work conducts analysis and provides practical insights on SFT.

Improving LLM Reasoning with RL Reinforcement Learning from Verifiable Rewards (RLVR) has emerged as a powerful paradigm for enhancing the reasoning capabilities of Large Language Models (Guo et al., 2025; OpenAI, 2024). Following initial successes, a significant body of open work has explored RLVR, primarily concentrating on specializing models for highly challenging single domains. Efforts such as Open-Reasoner-Zero (Hu et al., 2025), Skywork-OR1 (He et al., 2025), DeepScaler (Luo et al., 2025b), and SimpleRL (Zeng et al., 2025) have notably leveraged extensive mathematical data to achieve state-of-the-art performance on complex math benchmarks. Similarly, DeepCoder (Luo et al., 2025a) focused on RL for code generation tasks. While powerful within their specific areas, this domain-specific focus inherently limits the generalizability of the resulting models across the broader landscape of reasoning tasks. Concurrent works to our K2-Think development like General-Reasoner (Ma et al., 2025) and Nemotron-CrossThinker (Akter et al., 2025) have begun to explore broader domains for RL training. However, none of these works explore the added utility of test-time computation for improving the general reasoning capabilties of post-trained models.

Test Time Scaling Test-time scaling has been a major component of proprietary models released in recent years; such as o1 (OpenAI, 2024), Grok Heavy (xAI, 2025), Gemini 2.5 (Google DeepMind, 2025), and GPT-5 (OpenAI, 2025). However, with fairly little transparency about specific components and their overall effect. The closest work to ours is PlanGEN (Parmar et al., 2025), a multi-model framework for planning and reasoning combining a constraint model, a verification model, and a selection model to

guide inference-time algorithms including Best-of-N. By using constraint-guided iterative verification and a modified UCB-based selection policy, PlanGEN chooses the most suitable algorithm for each problem instance. Importantly, they use Best-of-N with verifiers on the *plans*: we use it for the generated *responses*.

Also related are general LLM-based hierarchical reasoning approaches, particularly those that operate with at least one level of hierarchy doing planning. Wang et al. (2024a) has a planning model provide high-level strategy while a solver model performs detailed reasoning. HyperTree Planning (Gui et al., 2025) models planning with a hypertree-structure, allowing LLMs to decompose planning queries into structured sub-tasks. Wang et al. (2025a) demonstrates a brain-inspired architecture with separate recurrent modules for high-level planning and low-level reasoning, showing that explicit separation of timescales improves performance on algorithmic reasoning tasks. Our novelty is to combine our "Plan-Before-You-Think" approach, a type of multi-LLM-hierarchical reasoning, with Best-of-N with verifiers (Cobbe et al., 2021) in order to return the best responses.

5 Discussion

5.1 Primary technical insights

Multiple domains are important for post-training. Following the findings from Guru (Cheng et al., 2025), there is a need to expand post-training to include more domains for general reasoning models. The effect of post-training, and the domains utilized, is nuanced. Domains commonly included in pre-training (Math, Code, and Science) broadly benefit from a variety of post-training data as the refinement of the model's chains of thought is supported by the knowledge it already has. However those domains with limited pre-training exposure–like Logic and Simulation tasks–only improve when they are included in the RL training pipeline. This indicates that using diverse, multi-domain datasets is critical for developing truly versatile reasoning models.

Test-time computation performance gains can be additive with the right combination. We find that two simple test-time computation procedures work well together: our "Plan-Before-You-Think" prompt restructuring in conjunction with Best-of-N scaling. Each individual method does improve over the K2-Think model but the largest gains in performance are seen when these components are combined. To our surprise, simply extracting a high level plan focused on the core concepts associated with the input and only sampling 3 candidate responses are sufficient to provide significant improvement.

"Plan-Before-You-Think" improves model performance while reducing token expenditure. By requiring the model to create a plan before initiating its reasoning process, we achieve two benefits: planning itself improves response quality, and response lengths are reduced by nearly 12%.

5.2 Looking forward

Empowering small models to "punch above their weight". With the complete K2-Think system, we demonstrate that a 32B-scale model, post-trained to produce long reasoning chains of thought, paired with relatively little test-time computation can endow the small model with capabilities that are competitive with models with orders of magnitude more parameters. Altogether our end-to-end reasoning system unlocks performance at the frontier of current open-source capabilities.

Beyond Open Source. We are extending the limit of our open-source activities beyond data, models and training artifacts. This expansion of our open-source efforts will now include deploying our full reasoning system for public use. We are publishing our test-time computation implementation as well. K2-Think is broadly available via API and an online web portal. In this we are opening avenues to explore how to best